

SHOULD BE REPLACED ON REQUIRED TITLE PAGE

Instruction

1. Open needed docx template (folder "title"/<your department or bach if bachelor student>.docx).
2. Put Thesis topic, supervisor's and your name in appropriate places on both English and Russian languages.
3. Put current year (last row).
4. Convert it to "title.pdf," replace the existing one in the root folder.

Contents

1	Introduction	7
1.1	Background	7
1.2	Research objectives	8
2	Literature Review	10
2.1	Software Architecture Foundations	10
2.2	Maintainability Foundations	11
2.3	Pattern–Tactic Interaction	13
2.4	Maintainability Tactics	14
2.4.1	Tactics for Modularity and Localized Change	15
2.4.2	Tactics for Analyzability and Observability	15
2.4.3	Tactics for Testability	16
2.4.4	Tactics for Understandability and Self-descriptiveness	16
2.4.5	Tactics for Reducing Coupling and Controlling Dependencies	17
2.4.6	Tactics to Mitigate Architectural Erosion and Technical Debt	17
2.4.7	Notes on Tactic Selection and Interaction	18
2.5	Maintainability Assessment Methods	18

2.6	LLMs for Code Refactoring and Architectural Tasks	19
2.6.1	Capabilities and Limitations of LLM-Based Refactoring	20
3	Methodology	22
3.1	Research Design	22
3.2	Architectural Taxonomy	23
3.3	Dataset	24
3.4	Pipeline Architecture	24
3.5	Repository Evidence Extraction	27
3.6	Maintainability Measurement	28
3.6.1	Maintainability Index	28
3.6.2	Supplementary Metrics	29
3.7	Architectural Tactic Catalog	30
3.8	LLM Configuration	31
3.9	Evaluation Method	31
4	Implementation	33
4.1	System Architecture	33
4.2	Technology Stack	35
4.3	Repository Dataset Preparation	35
4.4	Repository Artifact Extraction	37
4.4.1	File Tree Extraction	37
4.4.2	Import Graph Extraction	38
4.4.3	Code Signature Extraction	38
4.4.4	Structural Metrics Extraction	39
4.5	Architecture Detection Implementation	40
4.5.1	Prompt Construction	40

4.5.2	LLM Invocation Layer	41
4.5.3	Response Parsing	42
4.6	Static Analysis Implementation	42
4.7	Tactic Selection Implementation	43
4.8	Tactic Implementation Subsystem	45
4.8.1	Repository Indexing	46
4.8.2	BM25-Based File Retrieval	46
4.8.3	Planner Agent: File Selection	47
4.8.4	Patch Agent: Patch Generation	47
4.8.5	Patch Application and File State Management	48
4.8.6	Test Execution and Regression Detection	48
4.8.7	Self-Healing Loop	49
4.8.8	Iteration Control and Termination	50
4.8.9	Artifact Preservation	50
4.8.10	Validation	50
4.9	Maintainability Re-evaluation	51
4.10	Data Storage and Artifact Management	52
4.11	Automation and Execution Workflow	53
4.12	Reproducibility Considerations	54
5	Evaluation and Discussion	56
5.1	Overview	56
5.1.1	Research Questions	57
5.2	Study 1: Pipeline-Level Maintainability Evaluation	58
5.2.1	Pipeline Completion and Failure Rate	58
5.2.2	Maintainability Index Impact	59

5.2.3	Results by Architectural Tactic	60
5.2.4	Results by Architectural Style	61
5.2.5	Key Limitation Revealed by Study 1	62
5.3	Study 2: Architecture Detection Evaluation	63
5.3.1	Overall Classification Performance	63
5.3.2	Impact of Repository Evidence Type	64
5.3.3	Per-Class Classification Performance	65
5.3.4	Systematic Misclassification Patterns	66
5.3.5	Confidence Calibration	66
5.4	Study 3: Tactic Implementation on Architecture-Labeled Repositories	67
5.4.1	Tactic Implementation on Architecture-Labeled Repositories	68
5.5	Integrated Discussion	71
5.5.1	Architecture Detection Is Feasible but Boundary-Limited	72
5.5.2	Import Graphs Are the Most Valuable Architectural Signal	72
5.5.3	LLM Transformations Remain Code-Level Rather Than Architecture-Level	73
5.5.4	Script-Based Repositories Are the Strongest Use Case .	73
5.5.5	The Confidence Problem Propagates Across Studies . .	74
5.5.6	Failure Rate Requires Verification Mechanisms	74
5.6	Threats to Validity	75
5.6.1	Internal Validity	75
5.6.2	External Validity	75
5.6.3	Construct Validity	76
5.6.4	Conclusion Validity	76

CONTENTS 6

6	Conclusion	77
6.1	Summary of Contributions	77
6.2	Findings with Respect to Research Objectives	78
6.3	Limitations	81
6.4	Future Work	82
	Bibliography cited	84
A	Extra Stuff	89
B	Even More Extra Stuff	90

List of Tables

5.1	Pipeline outcome distribution across 162 repositories.	58
5.2	Maintainability Index descriptive statistics before and after LLM-implemented tactic application (N = 121).	59
5.3	Pipeline results grouped by selected architectural tactic.	60
5.4	Pipeline results grouped by detected architectural style.	61
5.5	Architecture detection accuracy and macro-F1 across models and prompt configurations. Best result in bold.	63
5.6	Accuracy change when each evidence type is added to the prompt.	64
5.7	Per-class architecture detection accuracy at P2 (file tree + import graph).	65
5.8	Modular monolith misclassifications at P2 (N = 33 per model). .	66
5.9	Mean LLM confidence for correct vs. incorrect predictions at P2.	67
5.10	Pipeline outcome distribution and MI statistics (N = 56).	68
5.11	ΔMI statistics for completed repositories (N = 42).	68
5.12	Results by tactic, architectural style, and repository size.	69

List of Figures

- 3.1 Architecture-aware LLM-based maintainability improvement pipeline. 27

Abstract

Software maintainability is a critical quality attribute that directly affects system evolution cost and reliability. Improving maintainability often requires architectural-level modifications demanding specialized expertise and significant manual effort. While Large Language Models (LLMs) have shown strong capabilities in code understanding and generation, their potential for systematic architecture-level software quality improvement remains largely unexplored.

This thesis investigates whether LLMs can support software maintainability improvement through architecture-aware tactic implementation. The work proceeds in three sequential empirical studies. The first study evaluates an end-to-end pipeline that detects repository architecture, selects an appropriate tactic from a predefined catalog, and implements it through LLM-generated code modifications, measuring the Maintainability Index (MI) before and after the intervention across 162 open-source Python backend repositories. The second study validates the architecture detection component independently by evaluating five LLMs across four context configurations on a manually labeled dataset of 57 repositories. The third study applies tactic implementation using validated architecture labels to isolate the effect of detection quality on implementation outcomes.

The end-to-end pipeline produces a statistically significant but modest overall MI improvement ($\overline{\Delta MI} = +0.48$, $p = 0.001$, $r = 0.28$), with 18.2% of completed repositories improving and a pipeline failure rate of 25.3%. Using validated architecture labels raises the improvement rate to 42.9% and the mean gain to +1.484, demonstrating that architecture detection quality propagates into downstream tactic selection.

The architecture detection study shows that the best configuration — Qwen3-coder-next with file tree and import dependency graph — achieves 70.2% accuracy and 0.65 macro-F1, exceeding the majority-class baseline by 12.3 percentage points. Import dependency graphs consistently improve classification across all five evaluated models, while code signatures degrade or do not improve performance. LLM confidence scores are poorly calibrated and provide no reliable signal for filtering uncertain predictions. All models systematically confuse modular monolith and layered architectures, a limitation that does not diminish with richer input.

The strongest maintainability improvements occur in small, script-based repositories where the Decomposability tactic can extract cohesive modules from monolithic files with near-zero MI scores. Repository size is the dominant moderating factor: on large repositories, individual file-level changes are absorbed by averaging and produce negligible aggregate MI changes. The implementation stage consistently operates at the code level rather than the architectural level, confirming the scale gap observed in prior LLM-based refactoring research.

These findings indicate that LLMs can support architecture-aware maintainability improvement as a targeted triage and transformation tool, particularly for structurally simple repositories with clear decomposition opportunities. Full autonomous architectural refactoring requires advances in architecture detection precision, implementation scope, and behavioral verification.

Keywords: software maintainability, architectural tactics, large language models, automated software improvement, static analysis, software architecture.

Chapter 1

Introduction

1.1 Background

Modern software systems increasingly rely on complex backend architectures that must remain maintainable throughout the development period. As systems grow in size and functionality, their internal structure tends to grow worse, leading to increased modification costs and technical debt [1]. Maintainability, defined in ISO/IEC 25010 as the ability of a system to be modified effectively [2], is therefore one of the most important quality attributes affecting the long-term sustainability of software.

One of the mechanisms for improving and maintaining maintainability at the architectural level is the use of architectural tactics – reusable architectural building blocks providing general architectural solutions for common issues pertaining to quality attributes [3]. Although these tactics are well described in the software architecture literature, their practical application in existing, imperfect, or evolving codebases remains inconsistent. Many real-world projects lack a clear architectural structure or have accumulated architectural erosion over years of de-

velopment, making it unclear how the targeted implementation of tactics affects maintainability in practice [4], [5].

At the same time, recent advances in large language models (LLMs) have opened up new possibilities for automatic program transformation. LLMs demonstrate the ability to refactor code, restructure modules, introduce design patterns, and enforce architectural rules, often while preserving system behavior [6]. This ability raises an important question: can LLMs be used to implement architectural tactics in existing backend systems, and if so, how will this affect maintainability?

Understanding this relationship is particularly relevant for open source software, where architecture quality varies significantly and where maintainability issues often arise due to rapid, community-driven development [4]. By using LLM to implement architectural tactics in real GitHub projects, researchers can empirically evaluate how structural improvements affect maintainability metrics, architectural consistency, and technical debt reduction.

The goal of this thesis is to research whether automated integration of architectural tactics using LLM can lead to measurable improvements in the maintainability of existing backend architectures.

1.2 Research objectives

To address the above issue, this thesis sets the following research objectives:

1. To identify a set of architectural tactics that directly influence maintainability as defined by ISO/IEC 25010.
2. Implement selected architectural tactics in existing open source backend projects using LLM to perform architectural transformations.

3. Evaluate the impact of LLM-generated architectural changes on maintainability using a combination of code-level and architecture-level metrics, static analysis tools, and architecture erosion indicators.
4. Compare maintainability before and after transformations using LLM, determining whether the tactics applied lead to measurable improvements.
5. Evaluate both the advantages and limitations of using LLM for architecture-level modifications, including their consistency, correctness, and ability to preserve behavior.
6. Formulate recommendations and guidelines for applying architectural tactics using LLM in real-world backend systems.

Chapter 2

Literature Review

This chapter provides a structured review of the existing research related to software architecture, maintainability, quality attributes, refactoring practices, technical debt, and architecture erosion. The objective of this review is to identify conceptual foundations, analytical methods, and empirical findings that support the investigation of how backend architectural styles influence software maintainability.

2.1 Software Architecture Foundations

Early research in software architecture established a conceptual level of reasoning that goes beyond individual components or algorithms. Software architecture is commonly defined as the organization of a system in terms of components, connectors, and their relationships, providing a high-level view of system structure [7]. This perspective shifts design from isolated implementation decisions toward reusable architectural structures governed by explicit constraints.

Architectural styles such as Pipes-and-Filters, Layered Systems, Event-based

architectures, and Repository-based architectures represent recurring structural solutions. Each style introduces specific invariants that restrict component interaction and shape system behavior. These structural constraints directly influence quality attributes such as modifiability, analyzability, and evolvability. In practice, many systems combine multiple architectural styles across different abstraction levels, which expands the architectural design space and introduces trade-offs between competing quality concerns [7].

In addition to structural organization, software architecture also captures design intent and rationale. A widely accepted model describes architecture as a combination of elements, their form, and the rationale behind design decisions [5]. This model emphasizes that architectural decisions are intentional and must be understood through multiple complementary views, including processing, data, and connector perspectives. Such multi-view descriptions are essential for understanding how architectural decisions affect system qualities over time [5].

Together, these foundational concepts establish software architecture as a primary design concern and provide the basis for reasoning about architectural change, degradation, and quality-oriented design [8].

2.2 Maintainability Foundations

Maintainability is widely recognized as a key quality attribute for long-lived software systems. It reflects the ease with which a system can be analyzed, modified, tested, and evolved. As systems increase in size and complexity, architectural decisions play a growing role in determining maintainability, often outweighing the impact of local code-level practices [8].

Empirical studies show that maintainability evolves over time rather than

remaining stable. Maintainability in open-source systems follows observable evolutionary trends, where mature systems tend to stabilize while significant degradation often coincides with major feature additions or architectural changes [1]. These findings also indicate that targeted refactoring can mitigate maintainability degradation, highlighting the importance of deliberate architectural improvement. Industrial evidence from Microsoft further confirms that developers regularly refactor to improve maintainability, but face significant effort and risk during these activities [9].

From a measurement perspective, standardized quality models provide a common basis for evaluation. The ISO/IEC 25010 quality model defines maintainability as a composite attribute consisting of modularity, reusability, analysability, modifiability, and testability [2]. This decomposition enables maintainability to be assessed through observable structural and behavioral properties rather than subjective judgment.

At the architectural level, these sub-characteristics are strongly influenced by system decomposition, dependency structure, and clarity of component responsibilities. For example, modular designs with clear interfaces support analysability and modifiability, while low coupling and high cohesion contribute to testability and controlled change impact [2], [8]. As a result, ISO/IEC 25010 provides a suitable theoretical foundation for analyzing the impact of architectural decisions on maintainability.

The Maintainability Index (MI), a composite metric combining Halstead Volume [10], Cyclomatic Complexity [11], and lines of code, has been widely adopted as a practical proxy for maintainability sub-characteristics [12]. However, its validity is debated, particularly because static analysis tools that compute related quality indicators exhibit very low agreement among each other, with

reported overlap of less than 0.4% across six tools [13]. This motivates a multi-metric approach to maintainability assessment, where MI is complemented by structural and dependency-level indicators.

2.3 Pattern–Tactic Interaction

Architectural patterns define high-level structural organization, while architectural tactics represent localized design decisions aimed at influencing specific quality attributes [8]. The interaction between patterns and tactics can be understood by examining the structural changes required to introduce a tactic into an existing architecture [14]. These changes range from minimal modifications within existing components to substantial restructuring that alters the original pattern.

A classification of tactic-related changes helps estimate implementation effort and architectural impact. The effectiveness and cost of a tactic depend on its compatibility with the underlying architectural pattern, meaning that tactics are not universally applicable [14]. Quality-driven architectural design further clarifies this relationship: the applicability of tactics is constrained by the structural properties of architectural patterns and their control- and data-flow semantics [3]. Empirical evidence shows that architectural patterns are rarely implemented without modification, and most real-world systems rely on pattern–tactic combinations rather than pure pattern implementations [4].

Architecture erosion poses a major threat to long-term maintainability. Common causes include inappropriate architectural changes, accumulation of technical debt, and uncontrolled growth in system complexity [15]. Various detection practices are used in industry and research, including dependency structure analysis,

architecture conformance checking, architectural smell detection, and visualization techniques [15]. Closely related to erosion, architectural drift refers to the gradual divergence between intended and implemented architecture. This divergence can be detected by comparing architectural models extracted from source code with the original design over multiple development iterations [16]. Both erosion and drift represent measurable architectural degradation that directly impacts maintainability.

Empirical mining of developer discussions reveals that architectural tactics have both positive and negative effects on quality attributes. Some tactics intended to improve one quality attribute may unintentionally harm maintainability, underscoring the need for systematic evaluation rather than relying solely on prescriptive guidance [17]. A recent systematic mapping by Márquez et al. catalogues a large set of architectural tactics together with their primary quality attribute impacts and provides a reusable structure for tactic-based reasoning [18]. Bogner et al. further show that modifiability tactics, in particular coupling reduction, are systematically addressed through service-oriented design patterns in both SOA and microservice architectures [19]. Rahmati and Tanhaei point out that misapplication of architectural patterns can lead to a measurable increase in maintenance effort, again emphasizing that patterns and tactics must be selected with respect to context [20].

2.4 Maintainability Tactics

This section summarizes architectural tactics that are explicitly intended to improve software maintainability. Tactics are grouped by the maintainability sub-characteristics they primarily support. Short descriptions and practical notes are

provided for each tactic, with citations to recent work that empirically discusses or applies them.

2.4.1 Tactics for Modularity and Localized Change

- **Modularization / Componentization.** Decompose the system into well-defined components with narrow, stable interfaces so that changes are localized and have minimal ripple effects; this reduces change impact and improves modifiability [20].
- **Adding Extension Points (Plug-in Architecture).** Design explicit extension points that allow new behavior to be added as plug-ins rather than modifying existing components; this enables adding features with lower modification cost [20].
- **Encapsulation and Information Hiding.** Hide internal details behind interfaces so that internal changes do not propagate to other parts of the system, improving modularity and lowering maintenance effort [3].
- **Deferred Binding / Late Binding.** Postpone binding of concrete implementations, for example via dependency injection or service registries, to allow easier replacement and reduce the cost of change [20].

2.4.2 Tactics for Analyzability and Observability

- **Logging and Audit Trails.** Record key events and state transitions to provide informative traces for debugging and root-cause analysis; well-structured logs reduce the time required to understand defects [17].

- **Impact Analysis and Change-Impact Tools.** Use systematic change-impact analysis to predict which components are affected by a planned change; this supports planning, risk assessment, and targeted testing [1].
- **Runtime Observability (Metrics, Tracing).** Expose meaningful runtime metrics and request traces so that system behavior can be inspected and understood without invasive code changes [13].

2.4.3 Tactics for Testability

- **Test Hooks and Facades.** Provide test-specific interfaces or facades that allow isolated testing of components without heavy setup, improving the speed and reliability of regression tests [20].
- **Design for Dependency Injection.** Structure components so that dependencies can be substituted in tests (mocks/stubs), reducing test setup complexity and enhancing automated test coverage [3].

2.4.4 Tactics for Understandability and Self-descriptiveness

- **Self-Descriptive Naming and Documentation.** Use clear naming conventions, inline documentation, and architecture-level documentation so that intent and component responsibilities are easier to grasp [20].
- **Standardization and Conventions.** Apply consistent patterns and coding/architecture conventions across the system to lower the learning curve for developers and reduce accidental architecture violations [4].

2.4.5 Tactics for Reducing Coupling and Controlling Dependencies

- **Layered Separation and Strict Interfaces.** Enforce clear layering and restrict cross-layer calls to control dependency directions, preventing unplanned coupling that hampers maintainability [7].
- **Dependency Inversion and Abstraction.** Depend on abstract interfaces rather than concrete implementations to minimize change propagation when implementations evolve [3].
- **Periodic Conformance Checking.** Automate checks that verify whether the implemented dependencies conform to the intended architectural constraints, in order to detect and prevent erosion early [15].

2.4.6 Tactics to Mitigate Architectural Erosion and Technical Debt

- **Continuous Architecture Monitoring.** Track architecture health indicators (e.g., smell density, cyclic dependencies) over time to identify erosion trends and trigger remediation before problems grow [15].
- **Refactoring with Small, Local Steps.** Prefer small, reversible refactorings (e.g., extract module, rename, move function) that gradually improve structure while reducing risk and cost [1].
- **Rollback and Reversion Support.** Provide mechanisms to revert changes safely so that risky modifications can be undone quickly if they harm maintainability or behavior [15].

- **Architecture-Conscious Code Reviews.** Include architecture-related checklist items in code review and accept only changes that preserve or improve architectural invariants [16].

2.4.7 Notes on Tactic Selection and Interaction

- **Pattern–Tactic Fit.** The cost and benefit of a tactic depend on the hosting architectural pattern: a tactic that is cheap in one pattern may be expensive or infeasible in another, so tactic selection must consider pattern constraints [14].
- **Trade-offs and Side Effects.** Some tactics improve one maintainability sub-characteristic while degrading another quality attribute; therefore, evaluation must consider both positive and negative impacts [17].
- **Tooling and Measurement.** A mix of static analysis tools, architecture conformance checks, and history-based metrics should be used to measure the effect of applied tactics, while being aware that tools may disagree and require triangulation [13].

2.5 Maintainability Assessment Methods

Assessing maintainability at the architectural level requires operational metrics that connect abstract quality attributes to concrete system properties. A structured framework decomposes maintainability into measurable architectural factors aligned with sub-characteristics such as testability, changeability, analyzability, and stability [20]. These factors translate architectural principles, such

as low coupling, high cohesion, and decomposability, into actionable evaluation criteria [8].

Static analysis tools are widely used to automate maintainability assessment. Comparative studies of popular tools reveal large differences in detection capabilities, precision, and rule coverage [13]. Extremely low agreement among tools indicates that assessment results depend strongly on tool selection, which highlights the need to combine multiple sources of evidence when evaluating architectural changes. As a practical consequence, recent empirical studies use the Maintainability Index together with structural metrics such as fan-in, fan-out, and dependency cycles to triangulate maintainability changes after refactoring.

2.6 LLMs for Code Refactoring and Architectural Tasks

Recent advances in large language models have enabled new forms of automated software maintenance, ranging from local code refactoring to repository-level architectural reasoning. Empirical evaluations show that LLM-based refactoring can preserve program behavior while improving maintainability-related attributes such as readability, analysability, and modifiability [6]. Common refactoring actions identified in these studies include renaming, decomposition, code simplification, and structural cleanup. These recurring patterns are strongly associated with improved maintainability, suggesting that large language models can serve as effective tools for applying maintenance-related refactorings at scale [6].

2.6.1 Capabilities and Limitations of LLM-Based Refactoring

Multi-agent frameworks such as MANTRA report high success rates in producing compilable, test-passing refactored code through contextual retrieval-augmented generation and multi-agent LLM collaboration [21]. However, empirical analysis of AI coding agents reveals a significant scale limitation: agentic refactorings are dominated by low-level, localized edits such as renaming, type changes, and small extractions, while higher-level design changes remain comparatively rare [22]. This is particularly problematic for architectural tactics, which by nature require coordinated changes across multiple modules.

Research into prompt engineering shows that while LLMs can identify a large fraction of refactoring opportunities, their performance in complex synthesis tasks remains limited [23]. Piao et al. explored bridging human expertise with LLM-based machine understanding for refactoring and found that structured prompts that include architectural context significantly improve output quality [24]. In security-related tactic implementation, Shokri and Khatchadourian reported high syntactic correctness but a sharp drop in semantic correctness, primarily due to the difficulty of coordinating inter-procedural changes across multiple classes [25]. A recent systematic literature review of LLM-based refactoring confirms that architecture-level transformations remain an open challenge, with the majority of LLM-based studies addressing only code-level transformations [26].

LLMs have also been applied to maintenance-oriented refactoring of Python ML projects, where multi-agent systems were used to perform a sequence of refactoring steps; the resulting work mostly focused on code-level patterns rather than full architectural reasoning [27]. A recent survey on generative AI for software architecture explicitly identifies architecture detection and tactic-based

refactoring as key open areas for future research [28].

Chapter 3

Methodology

This chapter describes the research methodology used to evaluate an architecture-aware LLM-based workflow for software maintainability improvement. The workflow integrates repository-level architecture detection, architectural tactic selection, and tactic implementation through LLM-generated code modifications, and is evaluated both as an end-to-end pipeline and at the level of individual components.

3.1 Research Design

This thesis follows a quantitative empirical research design grounded in the principles of evidence-based software engineering [8]. The primary objective is to evaluate whether LLMs can be used to improve software maintainability through architecture-aware tactic implementation.

The independent variable is the application of an architectural tactic selected and implemented by an LLM-based pipeline. The dependent variable is software maintainability, measured primarily through the Maintainability Index (MI) and

supplemented by structural and documentation-related metrics. Each repository serves as its own control: metrics are measured on the original codebase before the intervention and compared with metrics computed after the intervention.

The null hypothesis is:

$$H_0 : MI_{after} - MI_{before} = 0 \quad (3.1)$$

That is, the application of LLM-implemented architectural tactics does not produce a statistically significant change in the Maintainability Index. Since maintainability changes are not assumed to be normally distributed, non-parametric statistical methods are used for hypothesis testing.

3.2 Architectural Taxonomy

The pipeline operates over a taxonomy of three architectural styles applicable to Python backend repositories:

- **Script-based:** repositories centered around a small number of scripts, typically with a flat structure and limited modular decomposition.
- **Layered:** repositories organized around explicit technical layers, such as API/controllers, services, data access, models, and configuration, with dependencies flowing predominantly in one direction.
- **Modular monolith:** single deployable applications organized into feature- or domain-oriented modules without a strict technical layer hierarchy.

Microservice and event-driven styles are excluded from this taxonomy because they are distinguishable primarily through deployment-level evidence rather

than single-repository static structure. The distinction between layered and modular monolith is maintained because these styles imply different tactic-selection strategies and carry different implications for decomposition scope.

3.3 Dataset

Repositories were sourced from public Python backend projects on GitHub via the GitHub Search API. A multi-stage automated filtering process was applied to ensure dataset quality and relevance. The inclusion criteria required:

- a valid `requirements.txt` file confirming a Python dependency-managed project;
- presence of substantive Python source files (`.py`);
- backend-oriented project structure with evidence of active development.

The exclusion criteria removed repositories dominated by machine learning or data science artifacts, identified by the presence of Jupyter Notebooks or dominant dependencies such as `torch`, `tensorflow`, `scikit-learn`, `keras`, and `transformers`. This exclusion focuses the dataset on general-purpose backend systems rather than ML workflows or experimental code.

3.4 Pipeline Architecture

The architecture-aware maintainability improvement workflow is implemented as an automated *Pipes and Filters* pipeline. Each phase operates as an independent filter that consumes structured input artifacts and produces structured

output artifacts. This design supports intermediate artifact inspection, simplifies failure analysis, and allows individual stages to be reconfigured without affecting the rest of the pipeline.

The pipeline consists of the following sequential phases, illustrated in Figure 3.1:

1. **Repository cloning:** each candidate repository is shallow-cloned into an isolated temporary environment. This provides a clean, reproducible working state for subsequent analysis and modification.
2. **Baseline static analysis:** maintainability metrics are computed on the original, unmodified repository. This establishes the pre-intervention baseline used for comparison after tactic implementation.
3. **Repository artifact extraction:** structural evidence is extracted from the repository for use in LLM prompts. This includes the file tree, internal import dependency graph, code signatures, and aggregated structural metrics. These artifacts provide the LLM with a global view of the repository's organization and dependencies.
4. **Architecture detection:** the LLM classifies the repository into one of the three architectural styles defined in Section 3.2, using the extracted artifacts. The classification is accompanied by a confidence score, a list of supporting evidence observations, and a brief reasoning explanation, all returned in structured JSON format.
5. **Tactic selection:** given the detected architectural style and identified structural deficiencies, the LLM selects a maintainability-oriented architectural tactic from a predefined catalog. The model provides a justification linking

the chosen tactic to the expected quality improvement and an assessment of implementation risk.

6. **Tactic implementation:** the LLM applies the selected tactic through an iterative incremental modification process. In each iteration, the model proposes a small, self-contained code change in structured JSON format, specifying the target file and the intended modification. Each change is applied and validated for syntactic correctness before the next iteration begins. If validation fails, the step is retried or the pipeline is terminated for that repository.
7. **Post-intervention static analysis:** the same metric suite applied at baseline is recomputed on the modified repository, producing the post-intervention values used for comparison.
8. **Statistical analysis:** pre- and post-intervention metrics are compared for all repositories that completed the pipeline with paired measurements.

Repositories that fail at any stage — for example due to syntax errors introduced during implementation, unresolvable import errors, or LLM planning failures — are recorded as null outcomes and retained in the dataset to accurately report failure rates and robustness.

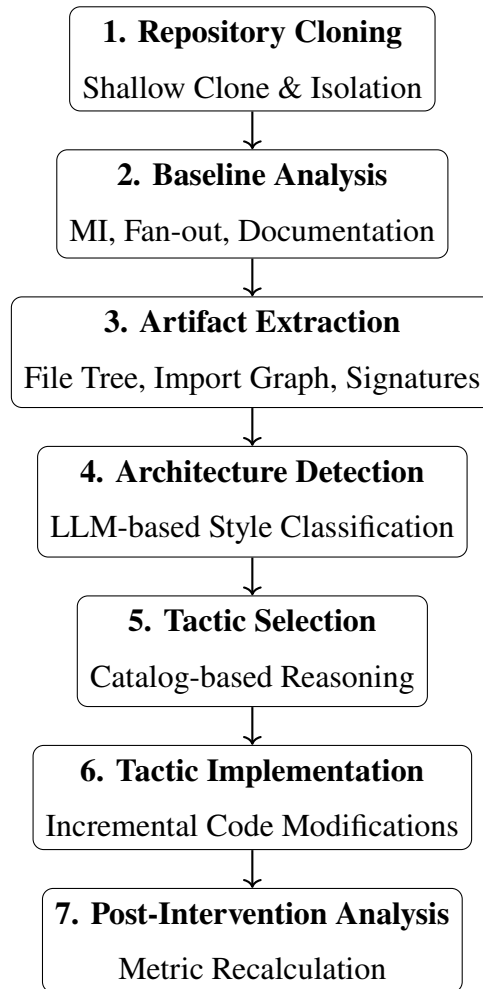


Fig. 3.1. Architecture-aware LLM-based maintainability improvement pipeline.

3.5 Repository Evidence Extraction

For each repository, four types of structural evidence are extracted and stored as artifacts:

1. **File tree.** An ASCII representation of the repository directory structure, excluding common non-source directories. This captures naming conventions, top-level organization, and framework-specific layout patterns.

2. **Import dependency graph.** Internal Python module dependencies extracted via AST parsing. Only imports between modules within the repository are retained; external library imports are excluded. The graph is stored as both a human-readable edge list and a structured JSON file.
3. **Code signatures.** Python class definitions with base classes, function signatures, and method signatures extracted using AST, without function bodies. This reduces context size while preserving interface-level information.
4. **Structural metrics.** Module count, average fan-out, maximum fan-in, dependency cycle detection flag, and total import edge count.

These four artifact types represent increasing levels of detail. The file tree captures static organization; the import graph captures dynamic structural relationships; signatures expose interface structure; metrics provide a compressed numerical summary. Depending on the pipeline stage and evaluation, different subsets of these artifacts are used.

3.6 Maintainability Measurement

Software maintainability is evaluated using a metric suite aligned with the ISO/IEC 25010 quality model [2].

3.6.1 Maintainability Index

The primary metric is the Maintainability Index (MI), computed using the Radon static analysis tool [12]. MI is a weighted composite of three components:

- **Halstead Volume**, which estimates program size and vocabulary complexity [10];
- **Cyclomatic Complexity**, which measures the number of independent execution paths through the code [11];
- **Lines of Code**, which captures the physical size of the module.

MI is reported on a scale from 0 to 100, where higher values indicate better maintainability. For each repository, the mean MI across all analyzed Python files is used as the aggregate repository-level value.

The maintainability change is computed as:

$$\Delta MI = MI_{after} - MI_{before} \quad (3.2)$$

A positive value indicates improvement, a negative value indicates degradation, and zero indicates no measurable change.

3.6.2 Supplementary Metrics

Because MI captures file-level complexity but is less sensitive to structural and coupling changes, additional metrics are collected to triangulate maintainability effects:

- **Average fan-out**: mean number of imports per module, used as a proxy for inter-module coupling.
- **Package count**: number of detected top-level packages, used as a coarse indicator of modular structure.

- **Docstring coverage:** proportion of functions and classes with docstrings, used as a documentation quality indicator.

These supplementary metrics are not treated as primary outcome variables but provide additional context for interpreting whether LLM-generated changes affected dimensions beyond code-level complexity.

3.7 Architectural Tactic Catalog

Tactic selection is grounded in the architectural tactic literature, particularly the catalog established by Bass et al. [8] and the systematic mapping by Márquez et al. [18]. The pipeline focuses on four maintainability-oriented tactics applicable to Python repositories:

- **Decomposability:** splitting monolithic files or modules into smaller cohesive units to improve analysability and modularity.
- **Reduced Coupling:** reducing inter-module dependencies through abstraction, mediation, or dependency inversion, targeting modifiability.
- **Localized Modification:** encapsulating change-prone logic to reduce the ripple effect of future modifications.
- **Deferred Binding Time:** externalizing configuration or deferring concrete implementation binding to improve flexibility.

The LLM selects the most appropriate tactic based on the detected architectural style, baseline structural metrics, and identified deficiencies such as high fan-out or monolithic files with low MI scores. The model provides a rationale linking the chosen tactic to the expected maintainability improvement.

3.8 LLM Configuration

All LLM calls in the pipeline use qwen3-coder-next:cloud through the Ollama API. This model was selected for its large 256k token context window, which allows the full repository context — including file tree, import graph, and code signatures — to be provided in a single prompt without truncation. A large context window is necessary for repository-level architectural reasoning, where cross-module dependencies must be visible simultaneously [24].

All LLM calls use the following parameters:

- temperature: 0.2, to reduce output variance and improve reproducibility;
- timeout: 300 seconds;
- retries: 2.

All prompts require structured JSON responses to enable automated parsing and downstream processing. Prompts for architecture detection include the allowed labels, taxonomy definitions, and a required output schema. Prompts for tactic selection and implementation include repository artifacts, baseline metrics, and task-specific instructions.

3.9 Evaluation Method

Completed repositories are categorized based on the sign of ΔMI :

- **Improved:** $\Delta MI > 0$;
- **Stable:** $\Delta MI = 0$;

- **Degraded:** $\Delta MI < 0$.

Repositories without valid post-intervention metrics are categorized as **failed** and excluded from paired statistical tests, but included in failure rate analysis.

The primary statistical test is the **Wilcoxon signed-rank test**, appropriate for paired non-normally distributed samples. The effect size is reported using the **matched-pairs rank-biserial correlation** (r), which distinguishes between statistical significance and practical importance.

Supplementary metrics are analyzed descriptively because the number of non-zero changes is typically too small for meaningful hypothesis testing.

Chapter 4

Implementation

This chapter describes the implementation of the experimental framework used in this thesis. The framework was designed to support two related evaluation modes. The first mode evaluates LLM-based repository architecture detection. The second mode executes the full architecture-aware maintainability improvement pipeline, including architecture detection, tactic selection, tactic implementation, and maintainability re-evaluation.

The implementation follows a modular pipeline architecture. Each processing stage is implemented as an independent component that consumes structured input artifacts and produces structured output artifacts. This design supports reproducibility, makes intermediate results inspectable, and allows individual stages to be replaced or reconfigured independently.

4.1 System Architecture

The framework follows the *Pipes and Filters* architectural pattern. Each filter performs one clearly defined task, such as repository artifact extraction, static

analysis, LLM invocation, tactic selection, or result aggregation. The output of each filter is stored as a file-based artifact and passed to the next stage.

The system consists of the following main subsystems:

- **Dataset preparation:** collects, filters, normalizes, and stores repository metadata.
- **Repository artifact extraction:** extracts file trees, import graphs, code signatures, and structural metrics.
- **Architecture detection:** builds prompts and invokes LLMs for architectural classification.
- **Static analysis:** computes maintainability-related metrics before and after modifications.
- **Tactic selection:** selects an architectural tactic based on detected architecture and repository metrics.
- **Tactic implementation:** applies LLM-generated code modifications incrementally.
- **Evaluation and artifact management:** stores raw outputs, intermediate artifacts, final datasets, and evaluation summaries.

The same implementation infrastructure is used in both evaluation modes. In the architecture detection evaluation, only the repository artifact extraction and architecture detection subsystems are used. In the pipeline-level maintainability evaluation, all subsystems are executed sequentially.

4.2 Technology Stack

The framework was implemented in Python because of its suitability for automation, availability of static analysis libraries, and compatibility with the selected target repositories. The core technology stack is summarized below:

- **Programming language:** Python 3.12;
- **Version control interaction:** Git;
- **Repository hosting platform:** GitHub;
- **Static analysis tool:** Radon;
- **LLM access layer:** Ollama API;
- **Data formats:** CSV, JSON, and plain-text artifacts.

CSV files are used for tabular datasets and aggregated results. JSON files are used for structured intermediate data, such as import graphs, LLM responses, and metric snapshots. Plain-text files are used for prompts, repository trees, and extracted signatures.

4.3 Repository Dataset Preparation

The dataset preparation subsystem is responsible for collecting candidate repositories, retrieving metadata, filtering repositories, and normalizing the resulting dataset.

Candidate repositories were obtained from GitHub using predefined search criteria. For each repository, metadata such as repository name, owner, primary

language, star count, fork count, watcher count, update timestamp, and size were retrieved using the GitHub REST API. Repository identity was represented by the full GitHub name in the form: owner/repository

Duplicate repositories were removed based on this full name.

For each candidate repository, the system performed a shallow clone:

```
git clone --depth 1 <repository_url>
```

Shallow cloning was used to reduce network traffic, processing time, and storage overhead. In later processing stages, repositories were deleted after artifact extraction or pipeline execution to ensure isolation and minimize local storage usage.

The filtering process retained repositories that satisfied the following conditions:

- presence of a requirements.txt file;
- presence of Python source files;
- backend-oriented project structure;
- absence of dominant machine learning dependencies;
- absence of notebook-oriented or experimental project structure.

Machine learning-heavy repositories were excluded by checking for dependencies and project artifacts associated with ML workflows, including packages such as torch, tensorflow, scikit-learn, keras, and transformers, as well as Jupyter Notebook files.

The normalized dataset contains repository metadata, structural metrics, architecture labels where available, and links to extracted artifacts.

4.4 Repository Artifact Extraction

Repository artifact extraction is a central part of the implementation because both architecture detection and tactic selection depend on repository-level context. The system extracts four categories of artifacts.

4.4.1 File Tree Extraction

The file tree extractor builds an ASCII representation of the repository directory structure. Common non-source directories are excluded, including:

- .git;
- __pycache__;
- virtual environment directories;
- build and distribution directories;
- IDE configuration directories;
- cache directories.

The file tree is depth-limited to avoid excessively large prompts. The resulting artifact is stored as:

artifacts/<owner>/<repository>/repo_tree.txt

This artifact captures project organization, top-level packages, framework-specific directories, and naming conventions.

4.4.2 Import Graph Extraction

The import graph extractor analyzes Python files using the Python AST module. For each Python module, the extractor identifies import statements and resolves internal dependencies between modules within the same repository.

The graph is represented as directed edges:

```
source_module -> target_module
```

External dependencies are excluded, since the goal is to capture internal architectural structure rather than library usage.

The import graph is stored in two formats:

```
artifacts/<owner>/<repository>/import_graph.txt
```

```
artifacts/<owner>/<repository>/import_graph.json
```

The text version is used in prompts, while the JSON version supports further analysis and reproducibility.

4.4.3 Code Signature Extraction

The code signature extractor reduces Python source files to architecture-relevant interface information. For each Python file, the extractor parses the file using AST and keeps:

- import statements;
- class definitions;
- base classes;

- function signatures;
- method signatures.

Function bodies are removed to reduce context size. The resulting artifact is stored as:

artifacts/<owner>/<repository>/repo_signatures.txt

This representation was used in the architecture detection evaluation to test whether code-level interface information improves classification accuracy.

4.4.4 Structural Metrics Extraction

In addition to textual artifacts, the system computes structural metrics from the repository, including:

- number of Python modules;
- average fan-out;
- maximum fan-in;
- number of import dependency edges;
- presence of dependency cycles;
- number of backend-related files.

These metrics are stored in the normalized dataset and can be included in LLM prompts or used during evaluation.

4.5 Architecture Detection Implementation

The architecture detection subsystem is responsible for constructing prompts, invoking LLMs, parsing responses, and storing predictions.

4.5.1 Prompt Construction

Architecture detection prompts are built from the extracted repository artifacts. Four prompt variants are supported:

- **P1:** file tree only;
- **P2:** file tree and import graph;
- **P3:** file tree, import graph, and code signatures;
- **P4:** file tree, import graph, code signatures, and structural metrics.

All prompt variants share the same system instruction. The instruction defines the task, allowed labels, architecture taxonomy, and required JSON response format.

The expected JSON response contains:

- `architecture_label`;
- `confidence`;
- `evidence`;
- `reasoning`.

A structured response format is required to enable automated parsing and metric computation.

4.5.2 LLM Invocation Layer

LLM access is implemented through a unified invocation layer built on top of the Ollama API. The invocation layer supports both local and cloud models. It implements:

- model selection;
- temperature configuration;
- timeout handling;
- retry logic;
- multi-key fallback for cloud models;
- response normalization.

For the architecture detection evaluation, the following cloud models were used:

- qwen3-coder-next:cloud;
- deepseek-v3.2:cloud;
- gemini-3-flash-preview:cloud;
- gemma4:31b-cloud;
- minimax-m2.7:cloud.

All calls were executed with:

- temperature = 0.2;

- timeout = 300 seconds;
- retries = 2.

The raw LLM responses and parsed predictions are stored in CSV files for later evaluation.

4.5.3 Response Parsing

Because LLM outputs may contain formatting deviations, the parser first attempts to parse the full response as JSON. If this fails, it extracts the first JSON-like substring from the response and parses it. If parsing still fails, the output is marked as invalid and the raw response is preserved for inspection.

Parsed architecture detection results are stored with the following fields:

- predicted label for each prompt variant;
- confidence value;
- parse status;
- raw response.

4.6 Static Analysis Implementation

The static analysis subsystem computes maintainability-related metrics before and after tactic implementation. The Radon tool is used to compute Maintainability Index values:

```
radon mi --json <repository_path>
```

The JSON output is parsed and aggregated at repository level. The primary aggregate metric is the average Maintainability Index across analyzed Python files.

In addition to MI, the system computes:

- package count;
- average fan-out;
- docstring coverage;
- README presence.

Static analysis results are stored separately for the original and modified repository states:

```
artifacts/static_analysis/BEFORE/<repository_name>/  
artifacts/static_analysis/AFTER/<repository_name>/
```

This separation enables direct before/after comparison and preserves metric snapshots for reproducibility.

4.7 Tactic Selection Implementation

The tactic selection subsystem uses the LLM to choose an architectural tactic based on:

- detected architecture;
- repository file tree;

- import graph;
- baseline maintainability metrics;
- structural deficiencies identified by the model.

The tactic catalog includes maintainability-oriented tactics such as:

- **Decomposability;**
- **Reduced Coupling;**
- **Localized Modification;**
- **Deferred Binding Time.**

The LLM is instructed to return a structured JSON response containing:

- selected tactic;
- rationale;
- expected impact;
- risk assessment;
- suggested implementation plan.

The selected tactic and justification are stored as intermediate artifacts. These artifacts allow later analysis of whether tactic selection aligns with repository structure and detected architecture.

4.8 Tactic Implementation Subsystem

The tactic implementation subsystem translates the selected architectural tactic into concrete repository changes through an iterative agentic loop [29]. The design departs from naive single-prompt approaches, in which the entire repository is sent to the LLM in one request and the model is expected to produce a complete rewrite. Such approaches are fragile: large context causes the model to lose structural focus, generated changes are difficult to validate atomically, and there is no mechanism for detecting or recovering from regressions.

The loop interleaves retrieval, generation, application, and automated testing, allowing the agent to make targeted, verifiable changes one step at a time. The overall flow is as follows:

1. Build a structural index of the repository;
2. retrieve the most relevant files using BM25 ranking;
3. invoke a *Planner Agent* to select the next file to modify;
4. invoke a *Patch Agent* to generate a replacement for that file;
5. apply the patch to the repository on disk;
6. run the test suite to detect regressions;
7. if a regression is detected, restore the previous file state and attempt self-healing;
8. commit the step if no regression occurs, and iterate.

This loop continues until the maximum number of iterations is reached, the agent issues an explicit **Stop** signal, or an unrecoverable failure occurs.

4.8.1 Repository Indexing

Before any modification is attempted, the subsystem builds a lightweight structural index of the repository [30]. For each Python file, the index stores the file path, the list of imported module names, the names of all defined classes, the names of all defined functions and methods, and the file size in characters. The index is constructed using the Python AST module without executing any code. Files that cannot be parsed due to syntax errors are indexed with metadata only [30].

The index serves two purposes. First, it provides a compact, queryable representation of repository structure that can be included in LLM prompts without transmitting full source code. Second, it is used as the document corpus for retrieval-based file ranking.

4.8.2 BM25-Based File Retrieval

To direct the agent’s attention toward files most relevant to the selected tactic, the subsystem ranks all indexed files using BM25, a standard term-frequency-based ranking algorithm widely used in information retrieval. The query is constructed from the tactic’s name, category, and description. Each file’s document is formed by concatenating its path, class names, function names, and import list [31].

BM25 scores are computed for all files, and the top- k files (with $k = 5$ by default) with positive scores are passed to the Planner Agent as a ranked candidate list. This retrieval step ensures that the agent starts from contextually relevant files rather than selecting blindly from the full repository, which is particularly important for large repositories where the tactic may only be applicable to a small

subset of files [31].

4.8.3 Planner Agent: File Selection

The Planner Agent is responsible for selecting the next file to modify. It receives the repository file tree, the full structural index, the BM25-ranked candidate list, the tactic description, and a history of steps already applied. Based on this context, the agent selects one file for inspection and modification, or issues a **Stop** signal if the tactic has been fully applied or no suitable candidate remains.

The agent's response is a structured JSON object containing exactly two fields: the action (`inspect_file` or `STOP`) and the selected file path. This constrained output format enables deterministic parsing and validation without relying on free-form text extraction. If the response cannot be parsed as valid JSON, or if the action or path fields are missing, the response is treated as a planning failure, and the iteration is retried up to a configurable maximum number of times before the pipeline aborts for that repository.

The planner is instructed to prefer files not yet modified in previous iterations, unless a prior change must be extended, and to avoid modifying `__init__.py` files to prevent inadvertent changes to package interfaces.

4.8.4 Patch Agent: Patch Generation

Once a target file has been selected, the Patch Agent is invoked to generate a replacement for that file. The agent receives the tactic description, the full current content of the target file, the history of applied steps, and optionally feedback from a failed test run if a previous patch attempt caused a regression.

The agent returns a structured JSON object specifying the action (`modify_file`

or `create_file`), the target file path, and the complete replacement file content. The requirement to return the full file content, rather than a diff or a partial edit, simplifies application and validation. A file is rejected if it exceeds a configurable line limit (400 lines by default) or if the path targets `__init__.py`.

The agent is instructed to make one focused change per step, to prefer small local refactors over large structural rewrites, and to preserve the original file's external interface wherever possible.

4.8.5 Patch Application and File State Management

After a patch has been validated structurally, the subsystem takes a snapshot of the current file state before applying any change. The snapshot is written to a per-iteration backup directory, keyed by iteration index and attempt number, so that rollback can be performed precisely for the affected file without reverting unrelated changes made in earlier iterations.

The patch is then applied by overwriting the target file with the content returned by the Patch Agent, or by creating a new file at the specified path if the action is `create_file`. After application, the in-memory repository state is updated to reflect the new file content, ensuring that subsequent iterations and prompts reference the current state of the repository rather than stale cached content.

4.8.6 Test Execution and Regression Detection

A key distinguishing feature of the implementation is automated test-based regression detection. After each patch is applied, the subsystem executes the repository's test suite using `pytest` with a configurable timeout (default: 300 seconds). The test output is parsed to extract the names of failed tests and the

total failure count.

Before the first modification is applied, a *test baseline* is captured by running the test suite on the original repository. The baseline records which tests were already failing before any LLM intervention, allowing the regression detector to distinguish between pre-existing failures and new failures introduced by a patch. A patch is considered to have caused a regression only if it introduces new test failures not present in the baseline; pre-existing failures are ignored.

Repositories without a detectable test infrastructure (no `pytest.ini`, `pyproject.toml`, `setup.cfg`, `tests/` directory, or `test_*.py` files) are processed with regression gating disabled. In such cases, patches are applied without test-based validation.

4.8.7 Self-Healing Loop

If a patch causes a regression, the subsystem does not immediately abort the iteration. Instead, it executes a self-healing procedure: the affected file is restored from the pre-patch snapshot, and the Patch Agent is re-invoked with the test failure output appended to the prompt as explicit feedback. The agent is instructed to propose a minimal fix that preserves the original tactic intent while avoiding the failing code path.

This self-healing loop runs for up to a configurable number of repair attempts per iteration (default: two). If all repair attempts cause regressions, the iteration is abandoned, the file is left in its pre-patch state, and the agent proceeds to the next iteration. The abandonment of an iteration is logged but does not terminate the overall pipeline; subsequent iterations may target different files.

4.8.8 Iteration Control and Termination

The agent loop continues until one of the following conditions is met:

- the maximum number of iterations is reached (default: five);
- the Planner Agent issues a Stop signal;
- the planner exceeds the maximum number of consecutive parse or validation failures.

Each completed iteration is recorded as an applied step, annotated with the action taken, the target file path, a short summary, and the test outcome. The history of applied steps is included in all subsequent prompts, providing the agent with a trace of prior modifications and enabling coherent multi-step refactoring sequences.

4.8.9 Artifact Preservation

All intermediate outputs are stored for reproducibility and post-hoc analysis. For each repository, the subsystem writes the structural index, the captured test baseline, one step artifact per completed iteration (containing the JSON patch), and one test result artifact per iteration (containing pytest output, failure names, regression flag, and success status). File-level backups are retained in the per-repository backup directory for the duration of the pipeline run.

4.8.10 Validation

After each modification step, the system validates syntactic correctness of the affected Python files. Validation includes parsing modified files and checking for syntax errors.

If validation fails, the repository is marked as failed unless the pipeline can recover through retry. Failures are retained in the final dataset as null outcomes.

The current implementation does not guarantee semantic behavior preservation. It does not systematically execute repository test suites before and after transformation. Therefore, the implementation should be interpreted as an experimental tactic-application mechanism rather than a production-ready refactoring tool.

A full test-based validation loop is considered future work.

4.9 Maintainability Re-evaluation

After tactic implementation, the static analysis subsystem is executed again on the modified repository. The same metric suite used during baseline analysis is recomputed. This ensures that pre- and post-intervention values are comparable.

The most important value computed at this stage is:

$$\Delta MI = MI_{after} - MI_{before} \quad (4.1)$$

The system stores:

- MI before;
- MI after;
- ΔMI ;
- fan-out before and after;
- docstring coverage before and after;

- package count before and after;
- execution status.

Repositories are classified as:

- **improved**, if $\Delta MI > 0$;
- **stable**, if $\Delta MI = 0$;
- **degraded**, if $\Delta MI < 0$;
- **failed/null**, if post-intervention analysis is unavailable.

4.10 Data Storage and Artifact Management

All experimental artifacts are stored in a structured directory hierarchy. The artifact management subsystem preserves both raw and processed data.

The stored artifacts include:

- repository metadata;
- normalized datasets;
- file trees;
- import graphs;
- code signatures;
- prompt templates;
- raw LLM responses;

- parsed LLM predictions;
- selected tactics;
- implementation logs;
- static analysis results;
- final evaluation datasets.

A typical repository artifact directory has the following structure:

```
artifacts/  
  owner/  
    repository/  
      repo_tree.txt  
      import_graph.txt  
      import_graph.json  
      repo_signatures.txt
```

Aggregated results are stored in CSV files. These files are used to compute evaluation metrics, confusion matrices, and statistical summaries.

4.11 Automation and Execution Workflow

The full pipeline is executed sequentially for repositories in the dataset. Sequential execution was chosen to simplify isolation and avoid interference between repository states.

For each repository, the system performs the following operations:

1. clone repository;
2. extract repository artifacts;
3. compute baseline static metrics;
4. run architecture detection;
5. run tactic selection;
6. apply LLM-generated modifications;
7. validate modified files;
8. recompute static metrics;
9. store results and logs;
10. delete local repository clone.

This workflow makes each repository run independent. Failures in one repository do not affect subsequent repositories.

4.12 Reproducibility Considerations

Several implementation decisions were made to improve reproducibility:

- repositories are identified by full name and commit hash where available;
- extracted artifacts are stored separately from transient cloned repositories;
- prompts are versioned as text templates;

- raw LLM outputs are stored before parsing;
- parsed results are stored in CSV format;
- static analysis outputs are preserved in JSON format;
- the pipeline can be re-run from intermediate artifacts without re-cloning repositories.

However, exact reproducibility is limited by the use of cloud LLMs. Model providers may update model implementations behind the same API tag. To mitigate this, the study stores raw outputs produced during the experimental run.

Chapter 5

Evaluation and Discussion

5.1 Overview

This chapter presents the evaluation of the architecture-aware LLM-based maintainability improvement workflow investigated in this thesis. The evaluation was conducted through three sequential empirical studies. Each subsequent study was motivated by limitations or open questions identified in the preceding one.

Study 1 executed the full architecture-aware pipeline on 162 Python backend repositories. The pipeline detected repository architecture, selected an architectural tactic, implemented code modifications, and re-measured maintainability. This study revealed that LLM-implemented tactics can produce statistically significant but modest improvements, and that architecture detection is a critical but insufficiently validated pipeline stage.

Study 2 was motivated by the lack of independent validation of the architecture detection stage in Study 1. The architecture detection component was evaluated separately using a manually labeled dataset of 57 repositories, five LLMs, and four evidence configurations. This study provided clear evidence

about which inputs are useful, which are harmful, and where systematic errors occur.

Study 3 applied architectural tactic implementation to the architecture-labeled 57-repository dataset. With ground-truth architecture labels available, this study tested whether more reliable architectural context produces better tactic implementation outcomes. The results were limited.

The three studies use different datasets because they answer different questions. Study 1 requires many repositories to assess pipeline robustness and aggregate maintainability impact. Study 2 requires manually labeled ground truth to evaluate classification accuracy. Study 3 uses the ground-truth labeled dataset to isolate the effect of architecture label quality on implementation.

5.1.1 Research Questions

The evaluation is organized around three research questions derived from the thesis research objectives:

- **RQ1:** Which LLMs and repository evidence types enable reliable architectural style classification of Python backend repositories?
- **RQ2:** To what extent does an LLM-implemented architectural tactic pipeline produce measurable maintainability improvements?
- **RQ3:** Which architectural styles and tactics are most amenable to LLM-driven maintainability improvement?

RQ1 is addressed in Study 2. RQ2 is primarily addressed in Study 1, and partially revisited in Study 3. RQ3 is addressed through both Study 1 and Study 3.

5.2 Study 1: Pipeline-Level Maintainability Evaluation

The first study evaluated the full architecture-aware maintainability improvement pipeline on 162 open-source Python backend repositories.

5.2.1 Pipeline Completion and Failure Rate

Out of 162 repositories, the pipeline produced paired before/after maintainability measurements for 121 repositories. The remaining 41 repositories produced null outcomes because the LLM introduced modifications that resulted in syntax errors, broken imports, or other issues preventing re-analysis. The overall failure rate was 25.3%.

Table 5.1 shows the distribution of pipeline outcomes.

TABLE 5.1
Pipeline outcome distribution across 162 repositories.

Outcome	N	Proportion
Improved ($\Delta MI > 0$)	22	13.6%
Stable ($\Delta MI = 0$)	92	56.8%
Degraded ($\Delta MI < 0$)	7	4.3%
Failed / null	41	25.3%
Total	162	100%

The dominant outcome was stability: 92 of 121 completed repositories showed no measurable Maintainability Index change. This does not mean that no code was modified. Inspection of preserved modification logs showed that even MI-stable repositories had an average of 4.7 code modification steps. However,

many modifications affected file-level complexity in ways that did not propagate to MI, or introduced compensating changes that offset each other.

5.2.2 Maintainability Index Impact

Table 5.2 summarizes the Maintainability Index before and after intervention across the 121 completed repositories.

TABLE 5.2

Maintainability Index descriptive statistics before and after LLM-implemented tactic application (N = 121).

Metric	MI before	MI after
Mean	65.11	65.59
Median	68.42	68.67
Standard deviation	19.17	19.22
Mean ΔMI	+0.48	
Median ΔMI	0.00	

A Shapiro-Wilk test confirmed non-normality of the delta distribution ($W = 0.34$, $p < 0.001$), justifying the use of a non-parametric test. A Wilcoxon signed-rank test on all 121 paired observations produced:

$$W = 360.0, \quad p = 0.001, \quad r = 0.28 \quad (5.1)$$

The result is statistically significant, but the effect size is small ($r = 0.28$). The median ΔMI of 0.00 indicates that improvements are concentrated in a subset of repositories rather than distributed uniformly.

Among the 22 repositories that improved, the mean improvement was:

$$\overline{\Delta MI}_{improved} = +2.89 \quad (5.2)$$

Individual improvements included:

- *get_subscribe*: $\Delta MI = +13.52$ (Decomposability tactic);
- *ps4-exploit-host*: $\Delta MI = +11.45$ (Decomposability tactic);
- *DXY-COVID-19-Crawler*: $\Delta MI = +9.99$ (Decomposability tactic).

These results address **RQ2**: the pipeline produces a statistically significant but modest overall maintainability improvement, concentrated in a subset of repositories where structure is low and decomposition is feasible.

5.2.3 Results by Architectural Tactic

Table 5.3 shows pipeline outcomes grouped by the architectural tactic selected by the LLM.

TABLE 5.3
Pipeline results grouped by selected architectural tactic.

Tactic	N	Null	Improved	Stable	Degraded	$\overline{\Delta MI}$
Decomposability	103	29	7	63	4	+0.55
Localized Modification	32	5	6	19	2	+0.13
Reduced Coupling	23	5	8	9	1	+0.70
Other	4	2	1	1	0	+0.67

Decomposability was the most frequently selected tactic, applied in 103 of 162 repositories. This reflects the composition of the dataset: most repositories

exhibited monolithic or loosely structured code, which makes decomposition a natural first-choice tactic.

Reduced Coupling showed the highest improvement rate: 8 of 18 completed cases improved, giving an improvement rate of 44.4%. This suggests that coupling-reduction transformations can be effective when the LLM can identify and modify dependency relationships without breaking execution.

Decomposability also had the highest null rate among completed cases. Decomposing unstructured code is technically more complex than introducing abstractions, which may explain why Decomposability produces both the largest gains and the most failures.

5.2.4 Results by Architectural Style

Table 5.4 shows pipeline outcomes grouped by the architectural style detected by the LLM.

TABLE 5.4
Pipeline results grouped by detected architectural style.

Architecture	N	Null %	Improved	Degraded	$\overline{\Delta MI}$
Script-based	71	29.6	5	0	+0.81
Modular monolith	83	19.3	15	7	+0.22
Layered	6	50.0	2	0	+0.94
MVC	2	50.0	0	0	0.00

Script-based repositories produced the highest mean improvement of +0.81 and contained all three largest individual gains. Script-based repositories often consist of large files with high Halstead Volume, which is directly reduced when files are split into cohesive modules.

Modular monoliths showed more modest mean improvement (+0.22) and accounted for all seven degradation cases. This asymmetry indicates that already-structured systems are sensitive to poorly targeted modifications: the LLM may inadvertently increase coupling or break internal boundaries in code that was already organized.

Layered repositories showed a high mean improvement (+0.94) with no degradations, but the sample was too small (three completed repositories) for reliable conclusions.

These results partially address **RQ3**: script-based repositories with decomposable monolithic files represent the most receptive targets for the current pipeline.

5.2.5 Key Limitation Revealed by Study 1

The initial pipeline experiment relied on LLM-reported confidence (median 0.95, range 0.85–0.98) as the only quality indicator for architecture detection. High concentration of certain tactics in specific architecture types suggested that detection may have been working, but this could not be verified without independent ground truth.

The 25.3% failure rate and the prevalence of neutral outcomes also suggested that implementation reliability was insufficient and that architecture detection errors may propagate into tactic selection. These limitations motivated the focused architecture detection study in Study 2.

5.3 Study 2: Architecture Detection Evaluation

Study 2 was conducted after Study 1 revealed that architecture detection is a critical but unvalidated stage of the pipeline. The study evaluated five LLMs and four context configurations on a manually labeled dataset of 57 Python backend repositories.

5.3.1 Overall Classification Performance

Table 5.5 shows accuracy and macro-F1 for all model-prompt combinations. The majority-class baseline, which always predicts *modular monolith*, yields 57.9%.

TABLE 5.5

Architecture detection accuracy and macro-F1 across models and prompt configurations. Best result in bold.

Model	P1		P2		P3		P4	
	Acc	F1 _m	Acc	F1 _m	Acc	F1 _m	Acc	F1 _m
Qwen3	.596	.53	.702	.65	.632	.52	.667	.57
MiniMax	.579	.55	.649	.61	.649	.55	.649	.56
Gemma4	.561	.48	.632	.55	.614	.47	.614	.45
DeepSeek	.544	.46	.614	.50	.614	.48	.632	.49
Gemini	.526	.47	.579	.49	.561	.41	.596	.45

The best configuration was Qwen3 with P2 (file tree + import graph): 70.2% accuracy, 0.65 macro-F1. This exceeds the majority-class baseline by 12.3 percentage points and demonstrates that LLM-based architecture detection can provide useful automated triage. However, it is not reliable enough to replace expert judgment, especially at architecture boundaries.

The most important pattern in the results is not model ranking but the consistent effect of evidence type: the delta between P1 and P2 is positive for all five models, while the delta between P2 and P3 is zero or negative for all five models.

This addresses **RQ1 (partial)**: LLMs can classify repository architecture at above-baseline accuracy when provided with file trees and import dependency graphs.

5.3.2 Impact of Repository Evidence Type

Table 5.6 shows the accuracy change when each evidence type is added.

TABLE 5.6
Accuracy change when each evidence type is added to the prompt.

Model	P1 → P2 (+import graph)	P2 → P3 (+signatures)	P3 → P4 (+metrics)
Qwen3	+10.5%	−7.0%	+3.5%
MiniMax	+7.0%	0.0%	0.0%
Gemma4	+7.0%	−1.8%	0.0%
DeepSeek	+7.0%	0.0%	+1.8%
Gemini	+5.3%	−1.8%	+3.5%

Adding the import dependency graph improved accuracy for every model. The import graph encodes inter-module dependencies, which directly reflect how a repository is structurally organized. File trees capture only naming hierarchy; import graphs expose coupling direction and internal modular boundaries.

Adding code signatures degraded or did not change performance for all models. Code signatures introduce a large volume of naming information that

may bias the model toward surface-level patterns such as class names containing Service, Repository, or Controller, which can be misleading when distinguishing between modular monolith and layered styles.

Adding structural metrics provided marginal and inconsistent gains.

The conclusion is unambiguous: the optimal evidence configuration for LLM-based architecture detection is file tree combined with import dependency graph. Code signatures should be omitted.

This fully addresses **RQ1**: import dependency graphs are the most valuable evidence type, while code signatures actively harm performance.

5.3.3 Per-Class Classification Performance

Table 5.7 shows per-class accuracy at P2.

TABLE 5.7

Per-class architecture detection accuracy at P2 (file tree + import graph).

Class (N)	Qwen3	MiniMax	Gemma4	DeepSeek	Gemini
Modular monolith (33)	.727	.636	.606	.515	.424
Layered (16)	.750	.688	.750	.750	.750
Script-based (8)	.750	.750	.750	.875	.750

The script-based and layered styles are detected reliably across all models, with minimum accuracy of 0.688. The modular monolith class is the primary source of classification errors, with accuracy ranging from 0.424 (Gemini) to 0.727 (Qwen3). Since modular monolith accounts for 57.9% of the dataset, errors in this class drive most of the overall accuracy differences between models.

5.3.4 Systematic Misclassification Patterns

Table 5.8 shows how modular monolith repositories were misclassified at P2.

TABLE 5.8
Modular monolith misclassifications at P2 (N = 33 per model).

Model	Correct	→ layered	→ script	→ other
Qwen3	24	8	1	0
MiniMax	21	7	2	3
Gemma4	20	9	3	1
DeepSeek	17	11	5	0
Gemini	14	14	4	1

The dominant error is modular monolith being classified as layered. This is a systematic pattern: all five models exhibit it, and it does not diminish significantly with richer input. The distinction between modular monolith and layered requires understanding whether packages represent technical responsibilities (presentation, service, data) or domain features (users, orders, products). This distinction is semantic and depends on naming intent, which is not fully visible from import topology alone.

5.3.5 Confidence Calibration

Table 5.9 compares mean confidence for correct and incorrect predictions at P2.

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories⁶⁷

TABLE 5.9
Mean LLM confidence for correct vs. incorrect predictions at P2.

Model	Correct	Incorrect	Δ
MiniMax	.838	.812	.026
Qwen3	.893	.876	.017
Gemma4	.918	.904	.014
DeepSeek	.853	.839	.014
Gemini	.916	.907	.009

The maximum observed difference between correct and incorrect confidence is 0.026, which is too small for practical filtering. Gemini reports the highest mean confidence despite having the lowest accuracy among the five models — a clear calibration inversion. This finding has a direct implication for the pipeline used in Study 1: the self-reported confidence values used to assess architecture detection quality in that experiment cannot be treated as accuracy indicators.

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories

Study 3 was motivated by the question of whether providing the pipeline with better-quality architecture information would improve implementation outcomes. After Study 2 produced manually validated architecture labels for 57 repositories, those labels were used as input to the tactic implementation stage, replacing the LLM’s self-reported classifications.

5.4.1 Tactic Implementation on Architecture-Labeled Repositories

This evaluation applies the tactic selection and implementation pipeline to the 57 manually labeled repositories from the architecture detection study, using validated architecture labels as input. The goal is to isolate tactic implementation quality from architecture detection errors.

Outcomes

Of 56 processable repositories, 14 (25.0%) failed during cloning. The remaining 42 produced paired measurements. Table 5.10 summarizes the outcome distribution and Table 5.11 the aggregate statistics.

TABLE 5.10
Pipeline outcome distribution and MI statistics (N = 56).

Outcome	N	Share of completed
Improved ($\Delta MI > 0$)	18	42.9%
Stable ($\Delta MI = 0$)	17	40.5%
Worsened ($\Delta MI < 0$)	7	16.7%
Failed / null	14	25.0% of all

TABLE 5.11
 ΔMI statistics for completed repositories (N = 42).

Metric	Value
Mean ΔMI (all completed)	+1.484
Mean ΔMI (improved only)	+3.716
Mean ΔMI (worsened only)	−0.653

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories 69

Compared to the initial pipeline experiment (Section 5.2), both the improvement rate (42.9% vs. 18.2%) and the mean gain (+1.484 vs. +0.48) are substantially higher. This is consistent with the hypothesis that validated architecture labels produce more coherent tactic selection.

Results by Tactic, Architecture, and Repository Size

Table 5.12 shows results broken down by the three primary grouping variables.

TABLE 5.12
Results by tactic, architectural style, and repository size.

Group	Category	N	Impr.	Worse.	$\overline{\Delta MI}$
Tactic	Decomposability	18	8	3	+2.33
	Localized Modification	14	9	2	+1.49
	Reduced Coupling	3	1	2	-0.16
	(none)	7	0	0	0.00
Architecture	Script-based	7	3	0	+5.62
	Modular monolith	26	9	5	+0.76
	Layered	9	6	2	+0.35
Size	Tiny (<10 files)	11	—	—	median Δ = 1.04
	Small (10–30 files)	9	—	—	median Δ = 0.76
	Medium (30–80 files)	12	—	—	median Δ = 0.05
	Large (>80 files)	10	—	—	median Δ = 0.00

Tactic. Localized Modification achieves the best risk-adjusted performance (9 improvements, 2 worsenings), while Decomposability is high-variance: it produces the largest gains (+21.98) but also generates more worsenings (3 cases) when new extracted modules have MI below the repository average.

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories70

Architectural style. Script-based repositories produce the highest mean improvement (+5.62) due to their few files and clear decomposition opportunities. Modular monoliths account for all five worsening cases, confirming that already-structured codebases are sensitive to structural interventions. Layered repositories show the highest improvement rate in count (6 of 9), suggesting that explicit layer boundaries provide a structured context for Localized Modification.

Repository size. Size is the dominant moderating variable. On tiny repositories, a single file split can raise the repository-level MI by over 20 points because the average is computed over only a few files. On large repositories (80+ files), individual changes are diluted by averaging and the median $|\Delta MI|$ is effectively zero. The correlation between Python file count and $|\Delta MI|$ is -0.13 , confirming a weak but consistent inverse relationship.

Key Improvement Mechanism

Across 12 of the 18 improved repositories, the core mechanism is the extraction of a new module achieving $MI = 100$. These modules contain cohesive, simple code: configuration objects, pure utility functions, or API clients with limited branching. A secondary pattern involves files with $MI = 0$ (Radon rank C): four repositories improved by at least 7 MI points after the LLM extracted part of a critically complex file, because the Halstead Volume component of MI decreases non-linearly with file size. Representative examples are:

- *kodekloudhub/webapp-color* ($\Delta MI = +21.98$): a single `app.py` was split into `app.py`, `color.py` ($MI = 100$), and `config.py` ($MI = 100$).
- *AutoLab-SAI-SJTU/Paper2Rebuttal* ($\Delta MI = +18.14$): `rebuttal_service.py` ($MI = 0$) was decomposed into three focused modules; the original file re-

covered to $MI = 55.6$.

- *HKUDS/FastCode* ($\Delta MI = +2.11$, 81 files): a rare large-repository success; retriever.py improved from $MI = 11.3$ to 78.4 and main.py from 4.9 to 77.0 through targeted extraction.

Failure and Degradation Analysis

Stable outcomes (17 cases) arise from three causes: generated files containing syntax errors that Radon excludes from the average (3 cases), planner failures in which the agent cannot produce a valid modification plan after repeated retries (7 cases), and no-op changes such as docstring additions that do not affect MI . Worsening outcomes (7 cases) occur when newly created files have MI below the existing repository average; all seven are bounded above -1.24 MI points and all occur in small or tiny repositories.

Technical pipeline failures include Ollama API rate limiting (HTTP 429), repeated planner JSON parse errors, stuck modification loops, and invalid step proposals. These failure modes indicate that multi-step agentic LLM pipelines require more robust planning validation and loop-termination strategies.

5.5 Integrated Discussion

The three studies together provide a multi-level view of what works and what is currently limited in LLM-based architecture-aware maintainability improvement.

5.5.1 Architecture Detection Is Feasible but Boundary-Limited

Study 2 shows that LLM-based architecture detection can produce above-baseline accuracy with a lightweight scriptable pipeline. No GPU or fine-tuning is required, and the full classification pipeline uses only AST-based extraction and standard cloud API access.

However, 70.2% accuracy means approximately 30% of repositories are misclassified. Most of these errors are modular monolith classified as layered. In the context of the pipeline, this error is particularly costly: a system that is domain-organized will receive layer-specific tactic recommendations designed for a technical layer hierarchy, which may be inappropriate or harmful.

This connects directly to the degradation cases observed in Study 1, where all seven degradations occurred in modular monolith repositories.

5.5.2 Import Graphs Are the Most Valuable Architectural Signal

The evidence type evaluation in Study 2 provides a clear design recommendation for any LLM-based architecture analysis system: include import dependency graphs, omit code signatures.

Import graphs encode structural dependencies between modules. They make visible the patterns that define architectural styles: whether dependencies flow in one direction (layered), whether modules are roughly symmetric (modular monolith), or whether there are few dependencies overall (script-based). This information is both inexpensive to extract and architecturally meaningful.

Code signatures, in contrast, added context volume without adding structural signal, and appear to have introduced misleading naming cues.

5.5.3 LLM Transformations Remain Code-Level Rather Than Architecture-Level

Studies 1 and 3 both show that the pipeline selects tactics at the architectural level but implements them through code-level modifications. Package count remained unchanged in all 123 repositories with paired metric data. Average fan-out changed in only 28 repositories, with a small mean change of approximately -0.016.

The Maintainability Index, which reflects file-level complexity through Halstead Volume and Cyclomatic Complexity, can capture these changes. However, architectural indicators such as module boundary structure, top-level package organization, and inter-module coupling patterns were largely unaffected.

This is consistent with the broader limitation observed in LLM-based refactoring research: even with repository-level context, LLMs tend to produce localized edits rather than restructuring. The implementation stage of the pipeline is therefore the primary bottleneck, independent of architecture detection quality.

5.5.4 Script-Based Repositories Are the Strongest Use Case

Across both Study 1 and Study 3, the strongest improvements occurred in script-based repositories where the Decomposability tactic was applied. These repositories typically consist of large monolithic files with high Halstead Volume and weak internal structure. Splitting such files into smaller cohesive modules directly reduces the inputs to the MI formula.

This represents a practical sweet spot: low-structure repositories offer clear decomposition targets, and even simple modularization produces measurable metric gains. The pipeline is most useful as a targeted tool for this type of repository

rather than as a universal architecture-level refactoring system.

5.5.5 The Confidence Problem Propagates Across Studies

Study 2 demonstrated that self-reported LLM confidence does not distinguish correct from incorrect predictions. The maximum observed difference between correct and incorrect confidence was 0.026. Yet Study 1 relied on self-reported confidence as the sole indicator of architecture detection quality.

This means the interpretation of Study 1 architecture-level results must be treated cautiously. The tactic selection in Study 1 may have been partially based on incorrect architectural classifications, and confidence values cannot be used retroactively to identify which classifications were reliable.

Future iterations of the pipeline should use confidence scores only with significant caution, or replace them with structural consistency checks or ensemble agreement across models.

5.5.6 Failure Rate Requires Verification Mechanisms

The 25.3% failure rate in Study 1 indicates that the current implementation mechanism is not reliable enough for autonomous deployment. Failures include syntax errors, broken imports, and structural changes that prevent re-analysis.

A production-ready implementation would require at minimum:

- automated test suite execution before and after modification;
- build and import validation;
- rollback capability;

- human review for high-risk transformations.

The current framework provides syntactic validation but not behavioral preservation. This is a fundamental gap that limits the applicability of the pipeline to experimental contexts.

5.6 Threats to Validity

5.6.1 Internal Validity

In Study 1, each repository serves as its own control, reducing cross-project variance. However, MI changes may be influenced by factors beyond the intended tactic, such as code formatting side effects or coincidental complexity changes during modification. A single LLM model was used without comparison to alternatives or manual implementation of tactics.

In Study 2, the ground truth labels were assigned by a single annotator. The modular monolith/layered boundary is inherently ambiguous, and a second annotator may draw the boundary differently for some repositories. Each prompt was executed once per model; the low temperature setting (0.2) reduces but does not eliminate stochastic variance.

5.6.2 External Validity

All three studies are limited to Python backend repositories from GitHub. The taxonomy covers three architectural styles; microservice, event-driven, and hexagonal architectures are not represented. Results may not generalize to other languages, enterprise systems, or more architecturally complex repositories.

5.6.3 Construct Validity

The Maintainability Index is a composite metric that does not capture all dimensions of architectural maintainability. It is sensitive to file-level complexity but less sensitive to modularity, architectural conformance, and inter-module dependency quality. Many meaningful modifications may not register as MI changes.

Architecture style is a high-level abstraction that some repositories may exhibit in hybrid form. Forcing repositories into a single class label may not reflect their actual structure.

5.6.4 Conclusion Validity

The datasets used in all three studies are relatively small. Study 2 uses 57 repositories; Study 1 uses 162 repositories with 121 completed. Statistical power is limited, and all results should be interpreted as early empirical evidence rather than definitive claims. All rankings and effect sizes are preliminary baselines.

Chapter 6

Conclusion

This thesis investigated whether Large Language Models can support software maintainability improvement through architecture-aware tactic implementation. The investigation was organized around three empirical studies that together cover the full workflow from architecture detection to tactic implementation and maintainability measurement. This chapter summarizes the findings with respect to the research objectives stated in Chapter 1, identifies the main limitations of the work, and proposes directions for future research.

6.1 Summary of Contributions

The thesis makes three primary contributions.

An empirical evaluation of LLM-based architecture detection. The study provides the first systematic multi-model evaluation of repository-level architectural style classification using LLMs. Five models were evaluated across four context configurations on a manually labeled dataset of 57 Python backend repositories. The evaluation yields clear design recommendations: import dependency

graphs are the most valuable evidence type, code signatures should be omitted, and LLM confidence scores are insufficiently calibrated to be used as reliability indicators.

An empirical evaluation of LLM-implemented architectural tactics. The thesis evaluates an architecture-aware maintainability improvement pipeline across 162 repositories and subsequently with validated architecture labels across 57 repositories. The results quantify the relationship between architecture label quality, tactic selection coherence, and measurable MI improvement, and identify repository size and initial MI distribution as dominant moderating factors.

A reusable pipeline architecture and artifact dataset. The pipeline implementation, labeled dataset, extracted repository artifacts, prompt templates, and raw model outputs are made publicly available, enabling replication and extension of the reported experiments.

6.2 Findings with Respect to Research Objectives

Objective 1: Identify architectural tactics that directly influence maintainability. The thesis grounds tactic selection in the catalog established by Bass et al. and systematically mapped by Márquez et al., focusing on four tactics applicable to Python backend repositories: Decomposability, Localized Modification, Reduced Coupling, and Deferred Binding Time. Empirical results confirm that Decomposability and Localized Modification produce measurable MI improvements in receptive repositories, while Reduced Coupling requires careful application to avoid introducing complexity in small codebases.

Objective 2: Implement selected tactics using LLMs. The pipeline successfully selects and implements architectural tactics in 74.7% of attempted

repositories. The implementation mechanism evolved across the three studies from a single-prompt approach to an agentic loop with BM25-based file retrieval, planner-based step selection, patch generation, syntactic validation, and test-based regression detection with self-healing. This design substantially reduces the risk of large uncontrolled modifications and provides per-step traceability.

Objective 3: Evaluate the impact using code-level and architecture-level metrics. The primary evaluation metric is the Maintainability Index computed by Radon, supplemented by fan-out, package count, and docstring coverage. The analysis reveals that MI is sensitive to file-level complexity changes but insensitive to top-level package organization changes. Package count remained unchanged across all evaluated repositories, and fan-out changed in fewer than 25% of cases, indicating that the pipeline’s structural impact is primarily local rather than architectural. This finding motivates the use of complementary file-level and module-level metrics in future evaluations.

Objective 4: Compare maintainability before and after transformations. The end-to-end pipeline produces a statistically significant positive MI shift ($W = 360.0$, $p = 0.001$, $r = 0.28$) on 162 repositories, with a mean ΔMI of $+0.48$. Using validated architecture labels increases the improvement rate from 18.2% to 42.9% and the mean gain from $+0.48$ to $+1.484$, directly demonstrating the influence of detection quality on implementation outcomes. The strongest improvements occur in small, script-based repositories: the top five individual gains range from $+7.93$ to $+21.98$ MI points, all arising from the extraction of cohesive modules from monolithic files with near-zero MI scores.

Objective 5: Evaluate advantages and limitations of LLMs for architecture-level modifications. The thesis identifies several clear advantages and limitations.

On the advantage side, LLMs can reason about repository-level structure, se-

lect contextually appropriate tactics, and generate code modifications that produce measurable maintainability improvements without requiring predefined transformation rules. The pipeline operates automatically without manual intervention and scales to tens of repositories.

On the limitation side, four issues are consistently observed. First, architecture detection errors propagate into tactic selection: when a modular monolith is misclassified as layered, the selected tactic may be inappropriate. Second, the implementation consistently operates at the code level rather than the architectural level, producing file splits and local extractions rather than module boundary restructuring. Third, the pipeline failure rate of 25.3% reflects brittleness in multi-step LLM code generation, including syntax errors in generated files, stuck planning loops, and malformed JSON responses. Fourth, LLM confidence scores are uncalibrated and cannot be used to filter unreliable predictions, which has direct implications for any downstream automation that consumes architecture detection output.

Objective 6: Formulate recommendations for applying architectural tactics using LLMs. The empirical results support the following recommendations:

- Use file tree combined with import dependency graph as the default evidence configuration for architecture detection. Code signatures should be omitted.
- Do not rely on LLM confidence scores to identify unreliable architecture predictions. Human review is necessary for all cases where the modular monolith and layered boundary is relevant downstream.
- Prioritize repositories with small file counts and low initial MI scores (par-

ticularly files with MI near zero) as primary targets for LLM-implemented decomposition tactics, as these offer the highest improvement potential.

- Apply Localized Modification in preference to Decomposability when the risk of MI degradation must be minimized. Decomposability is high-variance and should be reserved for clearly unstructured codebases.
- Integrate test-based regression detection into any production deployment of LLM-generated code modifications, and retain pre-modification file snapshots to support rollback.
- Supplement MI with file-level metrics when evaluating tactic impact on medium and large repositories, as repository-level MI averaging conceals individual file improvements.

6.3 Limitations

The conclusions of this thesis are bounded by several limitations that should be considered when interpreting the results.

Single annotator. Architecture ground truth labels were assigned by one annotator. The modular monolith and layered boundary is the hardest annotation decision, and inter-annotator agreement was not measured. Some repositories may genuinely exhibit hybrid characteristics.

Dataset scope. All three studies are restricted to Python backend repositories from GitHub. The taxonomy covers three architectural styles. Results may not generalize to other languages, larger enterprise systems, microservice or event-driven architectures, or non-backend domains.

Metric sensitivity. The Maintainability Index is sensitive to file-level complexity but insensitive to module boundary quality and inter-module coupling changes. Many meaningful structural improvements may not register as MI changes, particularly in medium and large repositories.

Absence of behavioral verification. The current pipeline validates syntactic correctness but does not guarantee semantic behavior preservation. Improvements in MI may coexist with undetected behavioral regressions, particularly in repositories without test suites.

Single LLM for implementation. The implementation pipeline uses Qwen3-coder-next exclusively. Comparisons with other models were not performed for the full pipeline, and results may vary with different models.

Preliminary nature. All three studies produce early feasibility evidence rather than definitive performance claims. Effect sizes are small to moderate, datasets are limited in size and diversity, and statistical power is constrained.

6.4 Future Work

The findings of this thesis point to several directions for future investigation.

Multi-annotator ground truth. Expanding the architecture detection dataset with labels from multiple annotators and measuring inter-rater agreement would establish a more reliable benchmark, particularly for the modular monolith and layered boundary.

Behavioral preservation verification. Integrating automated test suite execution as a mandatory pipeline stage, and requiring all modifications to pass the test suite before being committed, would transform the pipeline from an experimental refactoring mechanism into a behaviorally safe transformation tool.

Architecture-level metrics. Replacing or supplementing MI with metrics that capture module boundary quality, architectural conformance, and inter-module coupling would provide more discriminating evaluation of architectural tactic effects, especially for medium and large repositories.

Broader taxonomy and languages. Extending the architecture taxonomy to include microservice, event-driven, and hexagonal styles, and evaluating the pipeline on Java, TypeScript, or Go repositories, would establish the generalizability of the current findings.

Multi-model architecture detection. Systematically comparing LLM models on the detection task with a larger balanced dataset would clarify whether Qwen3’s advantage in modular monolith detection is model-specific or reflects general capability differences.

Hybrid detection approaches. Combining LLM-based classification with static structural thresholds — for example, using dependency graph metrics to disambiguate modular monolith and layered boundaries — may reduce the systematic misclassification rate at the most difficult boundary.

Adaptive context selection. Developing strategies that vary the evidence configuration based on repository size or detected structural characteristics — providing richer context for small repositories and compressed summaries for large ones — could improve both classification accuracy and implementation coherence.

Bibliography cited

- [1] A.-J. Molnar and S. Motogna, “A study of maintainability in evolving open-source software,” in *International Conference on Evaluation of Novel Approaches to Software Engineering*, Springer, 2020, pp. 261–282.
- [2] *ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model*, <https://iso25000.com/en/iso-25000-standards/iso-25010>, [Online; accessed 2025-12-04], 2023.
- [3] S. Kim, D.-K. Kim, L. Lu, and S. Park, “Quality-driven architecture development using architectural tactics,” *Journal of Systems and Software*, vol. 82, no. 8, pp. 1211–1231, 2009.
- [4] M. Kassab, M. Mazzara, J. Lee, and G. Succi, “Software architectural patterns in practice: An empirical study,” *Innovations in Systems and Software Engineering*, vol. 14, no. 4, pp. 263–271, 2018. doi: [10.1007/s11334-018-0319-4](https://doi.org/10.1007/s11334-018-0319-4)
- [5] D. E. Perry and A. L. Wolf, “Foundations for the study of software architecture,” *ACM SIGSOFT Software Engineering Notes*, vol. 17, no. 4, pp. 40–52, 1992. doi: [10.1145/141874.141884](https://doi.org/10.1145/141874.141884)

- [6] K. DePalma, I. Miminoshvili, C. Henselder, K. Moss, and E. A. AlOmar, “Exploring ChatGPT’s code refactoring capabilities: An empirical study,” *Expert Systems with Applications*, vol. 249, p. 123 602, 2024.
- [7] D. Garlan and M. Shaw, “An introduction to software architecture,” *Advances in software engineering and knowledge engineering*, vol. 1, no. 3, 1993.
- [8] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th. Addison-Wesley, 2021, isbn: 978-0136886099.
- [9] M. Kim, T. Zimmermann, and N. Nagappan, “An empirical study of refactoring challenges and benefits at Microsoft,” *IEEE Transactions on Software Engineering*, vol. 40, no. 7, pp. 633–649, 2014. doi: [10.1109/TSE.2014.2318734](https://doi.org/10.1109/TSE.2014.2318734)
- [10] M. H. Halstead, *Elements of Software Science*. Elsevier, 1977.
- [11] T. J. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976. doi: [10.1109/TSE.1976.233837](https://doi.org/10.1109/TSE.1976.233837)
- [12] P. Oman and J. Hagemeister, “Metrics for assessing a software system’s maintainability,” *Conference on Software Maintenance*, pp. 337–344, 1992.
- [13] V. Lenarduzzi, F. Pecorelli, N. Saarimaki, S. Lujan, and F. Palomba, “A critical comparison on six static analysis tools: Detection, agreement, and precision,” *Journal of Systems and Software*, vol. 198, p. 111 575, 2023.
- [14] N. B. Harrison and P. Avgeriou, “How do architecture patterns and tactics interact? A model and annotation,” *Journal of Systems and Software*, vol. 83, no. 10, pp. 1735–1758, 2010.

- [15] R. Li, P. Liang, M. Soliman, and P. Avgeriou, “Understanding architecture erosion: The practitioners’ perceptive,” in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, IEEE, 2021, pp. 311–322.
- [16] J. Rosik, A. Le Gear, J. Buckley, M. A. Babar, and D. Connolly, “Assessing architectural drift in commercial software development: A case study,” in *Software: Practice and Experience*, vol. 41, Wiley Online Library, 2011, pp. 63–86.
- [17] T. Bi, P. Liang, A. Tang, and X. Xia, “Mining architecture tactics and quality attributes knowledge in Stack Overflow,” *Journal of Systems and Software*, vol. 180, p. 111 005, 2021. doi: [10.1016/j.jss.2021.111005](https://doi.org/10.1016/j.jss.2021.111005)
- [18] G. Márquez, H. Astudillo, and R. Kazman, “Architectural tactics in software architecture: A systematic mapping study,” *Journal of Systems and Software*, vol. 197, p. 111 558, 2022. doi: [10.1016/j.jss.2022.111558](https://doi.org/10.1016/j.jss.2022.111558)
- [19] J. Bogner, S. Wagner, and A. Zimmermann, “Using architectural modifiability tactics to examine evolution qualities of service- and microservice-based systems,” *Computer Science – Research and Development*, vol. 34, pp. 187–237, 2019. doi: [10.1007/s00450-019-00402-z](https://doi.org/10.1007/s00450-019-00402-z)
- [20] Z. Rahmati and M. Tanhaei, “Ensuring software maintainability at software architecture level using architectural patterns,” *AUT Journal of Mathematics and Computing*, vol. 2, no. 1, pp. 81–102, 2021. doi: [10.22060/ajmc.2021.19232.1044](https://doi.org/10.22060/ajmc.2021.19232.1044)
- [21] Y. Xu, F. Lin, J. Yang, T.-H. Chen, and N. Tsantalis, “MANTRA: Enhancing automated method-level refactoring with contextual RAG and multi-agent

- LLM collaboration,” in *arXiv preprint arXiv:2503.14340*, 2025. arXiv: [2503.14340](#).
- [22] K. Horikawa, H. Li, Y. Kashiwa, B. Adams, H. Iida, and A. E. Hassan, “Agentic refactoring: An empirical study of AI coding agents,” in *arXiv preprint arXiv:2511.04824*, 2025. arXiv: [2511.04824](#).
- [23] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, and H. Liu, “Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study,” *Automated Software Engineering*, vol. 32, no. 1, 2025. doi: [10.1007/s10515-025-00500-0](#)
- [24] Y. C. K. Piao, J. C. Paul, L. Da Silva, A. M. Dakhel, M. Hamdaqa, and F. Khomh, “Refactoring with LLMs: Bridging human expertise and machine understanding,” in *arXiv preprint arXiv:2510.03914*, 2025. arXiv: [2510.03914](#).
- [25] S. E. Shokri and R. Khatchadourian, “IPSynth: Interprocedural program synthesis for software architecture recovery and architectural tactics detection,” in *arXiv preprint arXiv:2403.10836*, 2024. arXiv: [2403.10836](#).
- [26] S. Martinez, L. Xu, M. Elnaggar, and E. A. AlOmar, “Software refactoring research with large language models: A systematic literature review,” in *Journal of Systems and Software*, Elsevier, 2025, p. 112 762. doi: [10.1016/j.jss.2025.112762](#)
- [27] A. P. Pucho, A. M. Ferreira, E. J. R. Cirilo, and B. B. Cafeo, “Refactoring python code with llm-based multi-agent systems: An empirical study in ml software projects,” in *Simpósio Brasileiro de Engenharia de Software (SBES)*, SBC, 2025, pp. 678–684.

-
- [28] M. Esposito et al., “Generative ai for software architecture: Applications, trends, challenges, and future directions,” *Applications, Trends, Challenges, and Future Directions*, 2025.
 - [29] S. Yao et al., “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
 - [30] F. Zhang et al., “Repocoder: Repository-level code completion through iterative retrieval and generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484.
 - [31] S. Robertson and H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*. Now Publishers Inc, 2009, vol. 4.

Appendix A

Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.